

Tender Management API – Spring Boot REST Application

Overview

Develop a **Spring Boot REST API** for managing **Tender Bidding details** with **JWT-based authentication and authorization**.

The system supports two roles:

- **BIDDER**
- **APPROVER**

Users authenticate via `/login` to receive a **JWT token**, which must be included in all subsequent requests.

Authorization rules are based on the role embedded in the JWT token.

1. Database Models

1.1 RoleModel

Field Name	Datatype	Primary Key	Foreign Key	Comments
id	Integer	Yes	No	Auto Increment
rolename	String	No	No	Unique

1.2 UserModel

Field Name	Datatype	Primary Key	Foreign Key	Comments
id	Integer	Yes	No	Auto Increment
username	String	No	No	

companyName	String	No	No	
email	String	No	No	Unique
password	String	No	No	
role	Integer	No	Yes	References RoleModel(id)

1.3 BiddingModel

Field Name	Datatype	Primary Key	Foreign Key	Comments
id	Integer	Yes	No	Auto Increment
biddingId	Integer	No	No	Unique
projectName	String	No	No	Final value = "Metro Phase V 2024"
bidAmount	Double	No	No	
yearsToComplete	Double	No	No	
dateOfBidding	String	No	No	Current date in dd/MM/yyyy format
status	String	No	No	Default value = "pending"
bidderId	Integer	No	Yes	References UserModel(id)

2. Initial Database Data

Roles Table

id	rolename
1	BIDDER
2	APPROVER

Users Table

username	companyName	password	email	role
bidder1	companyOne	bidder123\$	bidderemail@gmail.com	1
bidder2	companyTwo	bidder789\$	bidderemail2@gmail.com	1
approver	defaultCompany	approver123\$	approveremail@gmail.com	2

3. Security Requirements

Implement **JWT-based authentication and authorization**.

JWT Rules

- Token must be passed in **Authorization header**
- Format:

Authorization: Bearer <JWT_TOKEN>

Endpoint Access Rules

Color	Access
 Red	Only BIDDER
 Blue	Only APPROVER
 Green	Both BIDDER and APPROVER

All endpoints except **/login** must be authenticated.

4. API Endpoints

4.1 Login

Endpoint

POST /login

Description

Authenticates user and returns a **JWT token**.

Request Body

```
{  
  "email": "bidderemail@gmail.com",  
  "password": "bidder123$"  
}
```

Success Response

Status Code

200 OK

Response

```
{  
  "jwt": "your_jwt_token",  
  "status": 200  
}
```

Error Response

400 Bad Request

If credentials are invalid.

4.2 Add Bidding (BIDDER only)

Endpoint

POST /bidding/add

Description

Creates a new bidding entry.

The **bidderId** must correspond to the authenticated bidder.

Request Body

```
{  
  "biddingId": 2608,  
  "bidAmount": 14000000.0,  
  "yearsToComplete": 2.6  
}
```

Success Response

Status Code

201 CREATED

Response

```
{  
  "id": 1,  
  "biddingId": 2608,  
  "projectName": "Metro Phase V 2024",  
  "bidAmount": 1.4E7,  
  "yearsToComplete": 2.6,  
  "dateOfBidding": "07/07/2023",  
  "status": "pending",  
  "bidderId": 1  
}
```

Error Response

400 Bad Request

4.3 List Biddings (Both Roles)

Endpoint

GET /bidding/list

Query Parameter

bidAmount

Example

/bidding/list?bidAmount=15000000

Description

Returns all bidding records where:

bidAmount > given value

Success Response

200 OK

Returns list of bidding records.

Error Response

400 Bad Request

If no records exist.

Response message:

no data available

4.4 Update Bidding Status

(APPROVER only)

Endpoint

PATCH /bidding/update/{id}

Description

Updates the status of a bidding.

Request Body

```
{  
  "status": "approved"  
}
```

Success Response

200 OK

```
{  
  "id": 1,  
  "biddingId": 2608,  
  "projectName": "Metro Phase V 2024",  
  "bidAmount": 1.4E7,  
  "yearsToComplete": 2.6,  
  "dateOfBidding": "07/07/2023",  
  "status": "approved",  
}
```

```
"bidderId": 1
}
```

Error Response

400 Bad Request

4.5 Delete Bidding (Conditional Access)

Endpoint

DELETE /bidding/delete/{id}

Authorization Rules

Access allowed only if:

① User role = **APPROVER**

OR

② User role = **BIDDER AND** is creator of the bidding

Success Response

204 NO CONTENT

Response message

deleted successfully

Error Responses

Condition	Response
Bidding id not found	400 Bad Request → "not found"
User not creator	403 Forbidden → "you don't have permission"

5. Validation Rules

If any validation fails:

HTTP 400 - Bad Request

Possible validations:

- Unique fields
 - Required fields
 - Role validation
 - Authentication failure
 - Authorization failure
-

6. Application Setup

Install Dependencies

```
bash install.sh
```

Run Application

```
mvn spring-boot:run
```

Run Tests

```
mvn clean test
```

7. Swagger Documentation

Enable Swagger API documentation at:

```
/v3/api-docs
```

8. Port Configuration

If port **8080** is already in use:

Kill process

```
fuser -k 8080/tcp
```

or

```
sudo service jenkins stop
```

OR change port in:

```
application.properties
```

Example

```
server.port=8082
```